

Build School 課程 – C# 基礎課程 08

Bill Chung
V2026.1 台北夏季班

請尊重講師的著作權及智慧財產權!

Build School 課程之教材、程式碼等、僅供課程中學習用、請不要任意自行散佈、重製、分享，謝謝

簡易進銷存

簡易進銷存系統

- 系統概要：
 - 管理貨品明細資料
 - 管理業務員明細資料
 - 管理進貨
 - 管理出貨
 - 其他查詢功能
 - 業務員業績查詢
 - 公司營業額與利潤查詢

簡化情境

- 為了簡化整個情境，我們作如下的假設
 - 一次進貨只能進一種貨品
 - 一次出貨只能出一種貨品
 - 沒有甚麼庫存調整這種事

資料庫需求

- 貨品基本明細檔 (Product)
- 業務員基本明細檔 (SalesMan)
- 進貨明細檔 (Procurement)
- 出貨明細檔 (Selling)
- 出貨來源檔 (SellingSource) – 用來儲存
Selling 中每一單的貨品是從哪幾張進貨單的庫存
出去的

LAB

讓我們從資料庫開始

開啟範例方案

- 方案名稱：BuildSchoolBizSystem

BizDataLibrary 專案

- 這個專案已經先產生了必要的 DbContext 與 Models

- (1) BizContext 類別代表一個和 DB 溝通的總單元，基本上被視為是 Data Access Layer (DAL)
- (2) 那些 DbSet<T> 的屬性可以視為是對應資料庫中的個別資料表，事實上當然不是這麼簡單，有時他也會代表 view 或其他

```
public partial class BizContext : DbContext
{
    0 個參考
    public BizContext(DbContextOptions<BizContext> options)
        : base(options)
    {
    }

    0 個參考
    public virtual DbSet<Procurement> Procurements { get; set; }

    0 個參考
    public virtual DbSet<Product> Products { get; set; }

    0 個參考
    public virtual DbSet<Salesman> Salesmen { get; set; }

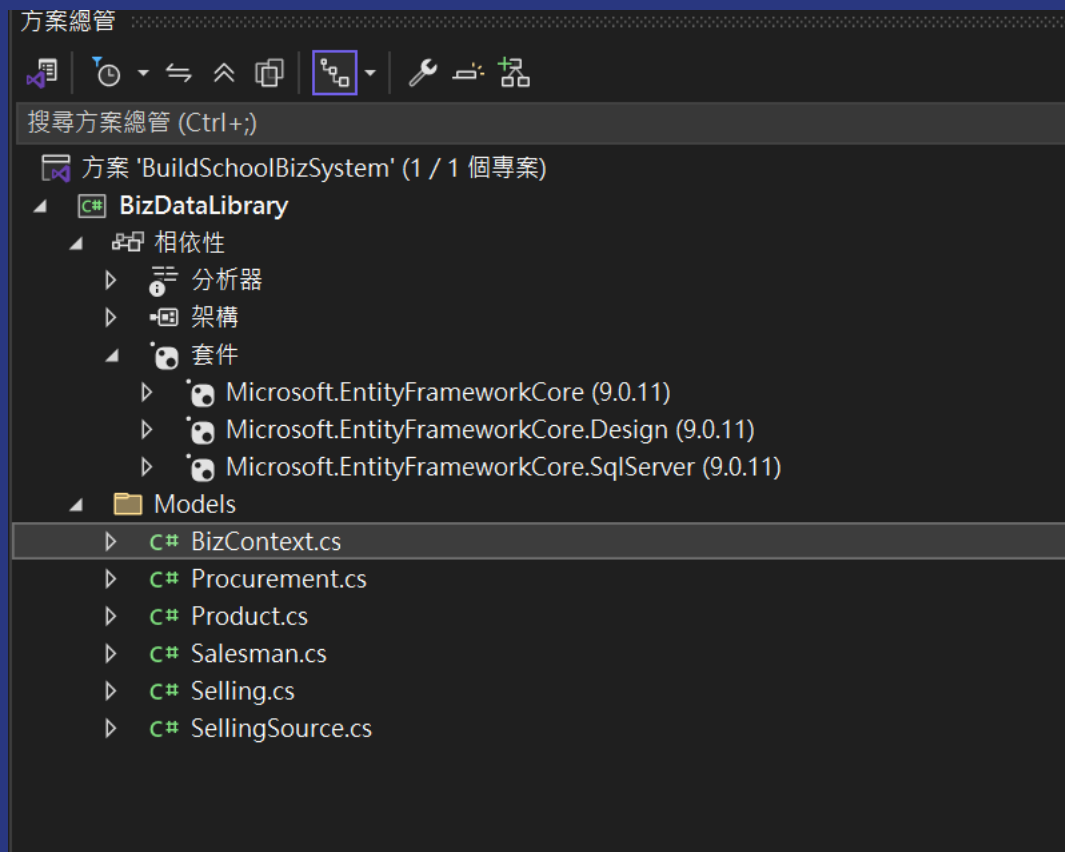
    0 個參考
    public virtual DbSet<Selling> Sellings { get; set; }

    0 個參考
    public virtual DbSet<SellingSource> SellingSources { get; set; }

    0 個參考
    protected override void OnModelCreating(ModelBuilder modelBuilder) ...

    1 個參考
    partial void OnModelCreatingPartial(ModelBuilder modelBuilder);
}
```

- (1) 而你看到的這些個別的類別，Procurement、Product、SalesMan、Selling 和 SellingSource 可以想像成是該資料表中的『一筆資料』
- (2) 所以他們的屬性命名和型別會符合該相對應資料表的欄位命名與型別



Repository

- Repository 是一種特殊的 pattern，它的作用主要在作為其他子系統和基本的 Data Access Layer (BizContext) 溝通的橋樑。
- Repository 的行為都是針對特定 Entity (如 Product Class) 的 CRUD 操作。
- 為了避免太複雜造成學習障礙，我們用最簡易的方式來寫 Repository。

如果你想更了解 Repository

- <http://kevintsengtw.blogspot.tw/2012/10/as-pnet-mvc-part1.html>
- <http://kevintsengtw.blogspot.tw/2012/10/as-pnet-mvc-part2-repository.html>
- <http://kevintsengtw.blogspot.tw/2012/11/as-pnet-mvc-part3-repository.html>

LAB

建立 Repositories

Repository 的基本組成

- void Create(資料型別)
- void Update(資料型別)
- void Delete(資料型別)
- IQueryable<資料型別> GetAll()
- 資料型別 Get(索引型別)
- void SaveChanges()
- 如果你有其他取得資料的操作，可以自行加入

- 請先在 BizDataLibrary 建立一個資料夾，名稱為 Repositories
- 對應每個資料的 Repository 都要建立在這個目錄下
- 接著依照後面兩張投影片建立 ProductRepository 和 SalesManRepository


```
public class ProductRepository
{
    private readonly BizContext _context;
    public ProductRepository(BizContext context)
    {
        _context = context;
    }
    public void Create(Product entity)
    {
        _context.Entry(entity).State = EntityState.Added;
    }
    public void Update(Product entity)
    {
        _context.Entry(entity).State = EntityState.Modified;
    }
    public void Delete(Product entity)
    {
        _context.Entry(entity).State = EntityState.Deleted;
    }
    public Product? Get(string partNo)
    {
        return _context.Products.FirstOrDefault(p => p.PartNo == partNo);
    }
    public IQueryable<Product> GetAll()
    {
        return _context.Products.OrderBy(p => p.PartNo);
    }
    public void SaveChanges()
    {
        _context.SaveChanges();
    }
}
```

```
public class SalesmanRepository
{
    private readonly BizContext _context;
    public SalesmanRepository(BizContext context)
    {
        _context = context;
    }
    public void Create(Salesman entity)
    {
        _context.Entry(entity).State = EntityState.Added;
    }
    public void Update(Salesman entity)
    {
        _context.Entry(entity).State = EntityState.Modified;
    }
    public void Delete(Salesman entity)
    {
        _context.Entry(entity).State = EntityState.Deleted;
    }
    public Salesman? Get(int jobNumber)
    {
        return _context.Salesmen.FirstOrDefault(s => s.JobNumber == jobNumber);
    }
    public IQueryable<Salesman> GetAll()
    {
        return _context.Salesmen.OrderBy(s => s.JobNumber);
    }
    public void SaveChanges()
    {
        _context.SaveChanges();
    }
}
```

討論

- 寫到這邊你發現甚麼？



LAB

讓我們重來一次

為 Repository 建立抽象類別

- 減少重複
- 降低遺漏的風險

討論

- 展開之前的兩個 Repository
- 哪些完全一樣？
- 哪些有點像又不完全一樣？

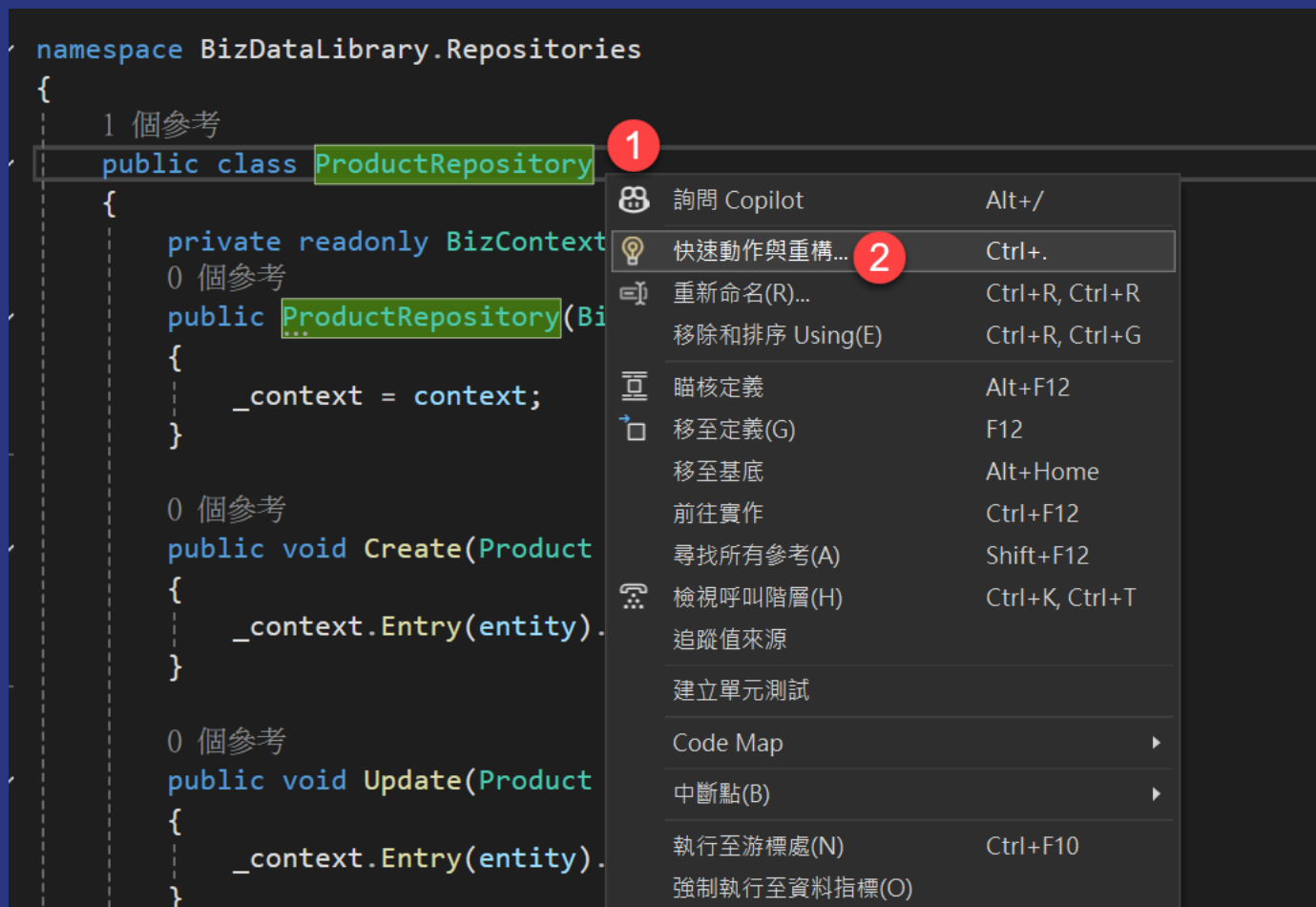


處理完全一樣的部分

Step 1 先處理完全一樣的部分

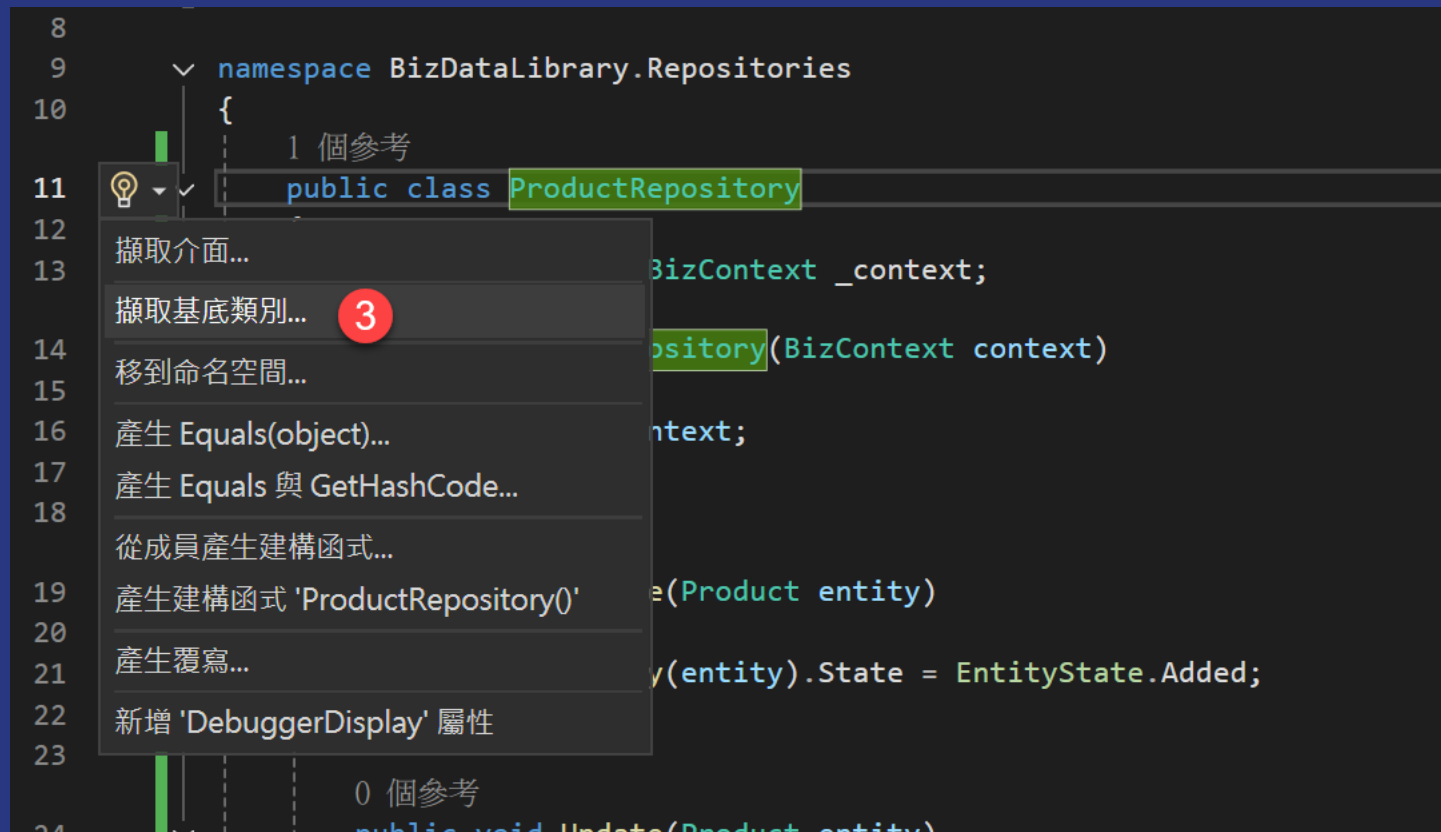
- 看來完全一樣的部分
 - `_context` 欄位
 - 建構式
 - `SaveChanges` 方法

擷取基底類別 (1)



焦點移到類別，按下滑鼠右鍵

擷取基底類別 (2)



選擇擷取基底類別

擷取基底類別 (3)

擷取基底類別

新的型別名稱:
BizRepository 1

產生的名稱:
BizDataLibrary.Repositories.BizRepository

選取目的地

☐ 新增至 current 檔案

☒ 新檔名(F): BizRepository.cs 2

選取成員: 3

成員	抽象化
☒ _context	
☐ Create(Product)	☐
☐ Delete(Product)	☐
☐ Get(string)	☐
☐ GetAll()	☐
☒ SaveChanges()	☐
☐ Update(Product)	☐

全選(S)

全部取消選取(D)

選取相依項(D)

選取公用(P)

4

確定 取消

修改型別名稱，選擇要擷取的成員

擷取基底類別 (4) 暫時的結果

```
public class BizRepository
{
    private readonly BizContext _context;

    0 個參考
    public void SaveChanges()
    {
        _context.SaveChanges();
    }
}
```

擷取基底類別 (5) 注意問題

```
public class ProductRepository : BizRepository
{
    0 個參考
    public ProductRepository(BizContext context)
    {
        _context = context;
    }

    0 個參考
    public void _context
    {
        _context
    }

    0 個參考
    public void Update(Product entity)
    {
        _context.Entry(entity).State = EntityState.Modified;
    }

    0 個參考
    public void Delete(Product entity)
    {
        _context.Entry(entity).State = EntityState.Deleted;
    }

    0 個參考
    public Product? Get(string partNo)
    {
        return _context.Products.FirstOrDefault(p => p.PartNo == partNo);
    }

    0 個參考
    public IQueryable<Product> GetAll()
    {
        return _context.Products.OrderBy(p => p.PartNo);
    }
}
```

class BizDataLibrary.Models.BizContext

使用 Copilot 加以描述

CS0122: 'BizRepository._context' 由於其保護層級之故，所以無法存取

顯示可能的修正 (Alt+Enter 或 Ctrl+.)

擷取基底類別 (6) 修改存取範圍與抽象類別

```
public abstract class BizRepository
{
    protected readonly BizContext _context;

    public void SaveChanges()
    {
        _context.SaveChanges();
    }
}
```

擷取基底類別 (7) 加入建構式

```
public abstract class BizRepository
{
    protected readonly BizContext _context;
    protected BizRepository(BizContext context)
    {
        _context = context;
    }
    public void SaveChanges()
    {
        _context.SaveChanges();
    }
}
```

可以使用【快速動作與重構】建立建構式
為什麼建構式的存取修飾詞是 `protected` ？

擷取基底類別 (8) 修改原有類別之一

- 修改 `ProductRepository` 的建構式，直接呼叫基底類別建構式

```
public class ProductRepository : BizRepository
{
    public ProductRepository(BizContext context) : base(context)
    {
    }
}
```


擷取基底類別 (9) 修改原有類別之二

- SalesManRepository 繼承 BizRepository
- 移除 `_context` 欄位
- 移除 `SaveChanges` 方法
- 修改 SalesManRepository 的建構式

```
public class SalesmanRepository : BizRepository
{
    0 個參考
    public SalesmanRepository(BizContext context) : base(context)
    {
    }
}
```

處理有點像又不太像的部分

泛型 Generic types

- .Net Framework 2.0後才出現泛型
- 泛型將型別參數的概念引進 .NET Framework 中，使得類別和方法在設計時，可以先行擱置一個或多個型別規格，直到用戶端程式碼對類別或方法進行宣告或執行個體化時再行處理
- `List<T>`就是一種泛型的設計
- 若不熟悉，請回去複習 CSharp07 的內容

泛型可以用在哪裡

- 泛型介面
 - `interface Itest<T>`
- 泛型類別
 - `class Test<T>`
- 泛型方法
 - `void Test<T> (T value)`
 - `T Test<T>()`
- 泛型委派
 - `delegate void Del<T>(T item)`

泛型條件約束

- 定義泛型類別時，可限制用戶端程式碼在執行個體化類別時的型別
- 使用 `where` 內容關鍵字指定條件約束
- `where T: struct`
 - 型別引數必須是實值型別
- `where T : class`
 - 型別引數必須是參考型別

- `where T : new()`
 - 型別引數必須擁有公用的無參數建構函式
 - 將 `new()` 條件約束與其他條件約束一起使用時，一定要將其指定為最後一個
- `where T : <base class name>`
 - 型別引數必須本身是指定的基底類別，或衍生自該類別
- `where T : <interface name>`
 - 型別引數必須本身是指定的介面，或實作該介面
- `where T : U`
 - 提供給 `T` 的型別引數必須是（或衍生自）提供給 `U` 的引數

在哪裡用泛型？

- 前面實作的兩個 `Repository`，哪些部分最有可能用泛型替代？

Step 1

(1) 先把抽象類別宣告為泛型型別

```
public abstract class BizRepository<T> where T : class
```


Step 2

- (1) 以泛型處理有點像又不太像的部分 (加入以下程式碼到 BizRepository class

```
public void Create(T entity)
{
    _context.Entry(entity).State = EntityState.Added;
}

public void Update(T entity)
{
    _context.Entry(entity).State = EntityState.Modified;
}

public void Delete(T entity)
{
    _context.Entry(entity).State = EntityState.Deleted;
}

public abstract IQueryable<T> GetAll();
```

提示 : Entry 其實是個泛型方法

```
public void Create(T entity)
{
    _context.Entry(entity).State = EntityState.Added;
}
```



```
public void Create(T entity)
{
    _context.Entry<T>(entity).State = EntityState.Added;
}
```

Step 3

- (1) 修改 ProductRepository ，讓他繼承 BizRepository<T>
- (2) 刪除 Create, Update 和 Delete 方法
- (3) 修改 GetAll 為 override 方法

```
public class ProductRepository : BizRepository<Product>
{
    public ProductRepository(BizContext context) : base(context)
    {
    }

    public Product? Get(string partNo)
    {
        return _context.Products.FirstOrDefault(p => p.PartNo == partNo);
    }
    public override IQueryable<Product> GetAll()
    {
        return _context.Products.OrderBy(p => p.PartNo);
    }
}
```

繼續完成其他的 Repository

- 依照前面的要領完成其他的 Repository
- 包含修改之前的 SalesManRepository
- 請注意 OrderBy 的部分使用主索引排序

```
public class SalesmanRepository : BizRepository<Salesman>
{
    0 個參考
    public SalesmanRepository(BizContext context) : base(context)
    {
    }

    0 個參考
    public Salesman? Get(int jobNumber)
    {
        return _context.Salesmen.FirstOrDefault(s => s.JobNumber == jobNumber);
    }

    1 個參考
    public override IQueryable<Salesman> GetAll()
    {
        return _context.Salesmen.OrderBy(s => s.JobNumber);
    }
}
```

討論

- 在這個些修改的過程中，你們獲得的心得是甚麼？
- 各組討論十五分鐘，並上台報告三到五分鐘。
- 這個課題很重要，請嚴肅面對。



能怎麼改進 GetAll ?

```
public override IQueryable<Salesman> GetAll()  
{  
    return _context.Salesmen.OrderBy(s => s.JobNumber);  
}
```

```
public override IQueryable<Product> GetAll()  
{  
    return _context.Products.OrderBy(p => p.PartNo);  
}
```



如果可以解決這兩個問題

解決第一個問題

- 要是可以 `Context.<T>` 就好了
- 使用 `DbContext.Set<TEntity>` 方法 來解決這問題。
- 這個 `method` 的作用是利用 `TEntity` 找到在 `DbContext` 中相對應的 `DbSet<TEntity>` 屬性。
- <https://learn.microsoft.com/en-us/dotnet/api/microsoft.entityframeworkcore.dbcontext.set?view=efcore-9.0>

解決第二個問題

- 先查詢 `IQueryable<T>.OrderBy` 是如何宣告的。
 - <https://learn.microsoft.com/zh-tw/dotnet/api/system.linq.queryable.orderby?view=net-9.0>
 - 這個方法的實作位在 `System.Linq.Queryable` class

```
public static IObservable<TSource> OrderBy<TSource, TKey>
    (this IQueryable<TSource> source,
     Expression<Func<TSource, TKey>> keySelector);
```


- (1) 所以理論上我們可以這樣多載 GetAll，讓排序的語法由外部決定。
- (2) 請不要將這個程式碼加入，我們將會在抽象類別裡撰寫這個方法。

```
public override IQueryable<Selling> GetAll()
{
    return _context.Salesman.OrderBy((x) => x.JobNumber);
}

public IQueryable<Salesman> GetAll<TKey>(Expression<Func<Salesman,TKey>> keySelector)
{
    return Context.Set<Salesman>().OrderBy(keySelector);
}
```

LAB

在 BizRepository 多載 GetAll

(1) 在 BizRepository 加入以下方法

```
public IQueryable<T> GetAll<TKey>(Expression<Func<T, TKey>> keySelector)
{
    return _context.Set<T>().OrderBy(keySelector);
}
```

成為泛型類別之路

- 應該宣告為泛型類別應具備下列兩個條件之一
 1. 實作泛型介面或是繼承泛型類別
 2. 該泛型類別會使用在欄位、屬性或索引子其中之一
- **BizRepository** 並不具備以上條件

LAB

修正泛型部分

如何修正？

- **BizRepository** 拿掉型別的泛型宣告，將<T>宣告移動到方法上宣告
- **ProductRepository** 與 **SalesManRepository** 繼承部分的泛型宣告移除，同時移除無參數的 **GetAll()**

```
public abstract class BizRepository
{
    protected readonly BizContext _context;
    protected BizRepository(BizContext context)
    {
        _context = context;
    }
    public void SaveChanges()
    {
        _context.SaveChanges();
    }

    public void Create<T>(T entity) where T : class
    {
        _context.Entry<T>(entity).State = EntityState.Added;
    }

    public void Update<T>(T entity) where T : class
    {
        _context.Entry(entity).State = EntityState.Modified;
    }
    public void Delete<T>(T entity) where T : class
    {
        _context.Entry(entity).State = EntityState.Deleted;
    }

    public IQueryable<T> GetAll<T,TKey>(Expression<Func<T,TKey>> keyselector) where T : class
    {
        return _context.Set<T>().OrderBy(keyselector);
    }
}
```

```
public class SalesmanRepository : BizRepository
{
    public SalesmanRepository(BizContext context) : base(context)
    {
    }

    public Salesman? Get(int jobNumber)
    {
        return _context.Salesmen.FirstOrDefault(s => s.JobNumber == jobNumber);
    }
}
```

```
public class ProductRepository : BizRepository
{
    public ProductRepository(BizContext context) : base(context)
    {
    }

    public Product? Get(string partNo)
    {
        return _context.Products.FirstOrDefault(p => p.PartNo == partNo);
    }
}
```


LAB

處理 Get method

如何修正？

- 兩個 `Get` 參數不一樣，但 `FirstOrDefault` 的呼叫可以外移
- `DbContext.Set<T>` 的回傳值是 `DbSet<T>`，而 `DbSet<T>` 實作了 `IQueryable<T>`
- `GetAll` 的 `OrderBy` 也可以外移
- 這些外移都會移動到 `Service` 層

修改 BizRepository.GetAll

- 將 GetAll 修改如下

```
public IQueryable<T> GetAll<T>() where T : class
{
    return _context.Set<T>();
}
```

移除無用的程式碼

- 此時 `SalesManRepository` 和 `ProductRepository` 都沒有用了
- 整個註解掉或乾脆刪除

移除 abstract 宣告

- 最終只剩下一個 BizRepository，所以它沒有必要是 abstract class

```
public abstract class BizRepository
```



```
public class BizRepository
```

建構式存取修飾改為 public

```
public BizRepository(BizContext context)
{
    _context = context;
}
```

欄位存取修飾改為 private

```
private readonly BizContext _context;
```

LAB

依賴抽象

改變依賴

- **BizRepository** 依賴了 **BizModel**，這表示如果是多個資料庫就得建立多個 **Repository**
- 改變依賴為 **BizContext** 的基底類別 **DbContext**

```
private readonly BizContext _context;  
public BizRepository(BizContext context)  
{  
    _context = context;  
}
```



```
private readonly DbContext _context;  
public BizRepository(DbContext context)  
{  
    _context = context;  
}
```

應用 Library

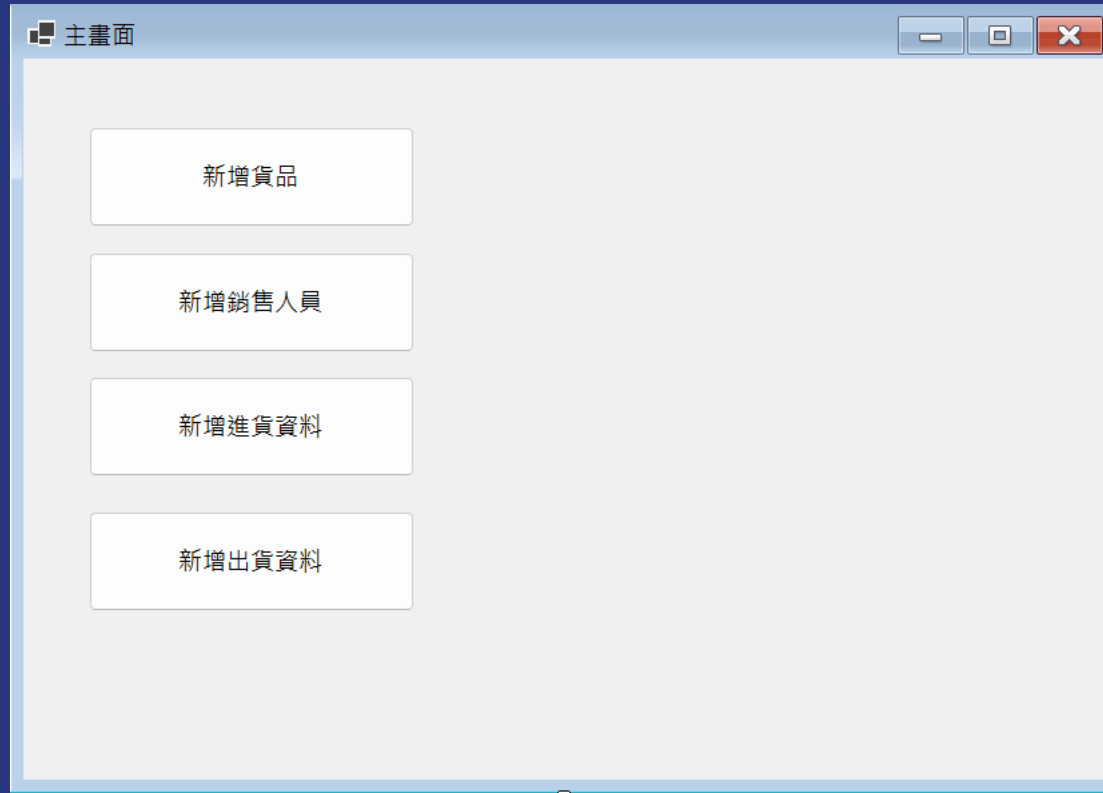
LAB

建立畫面

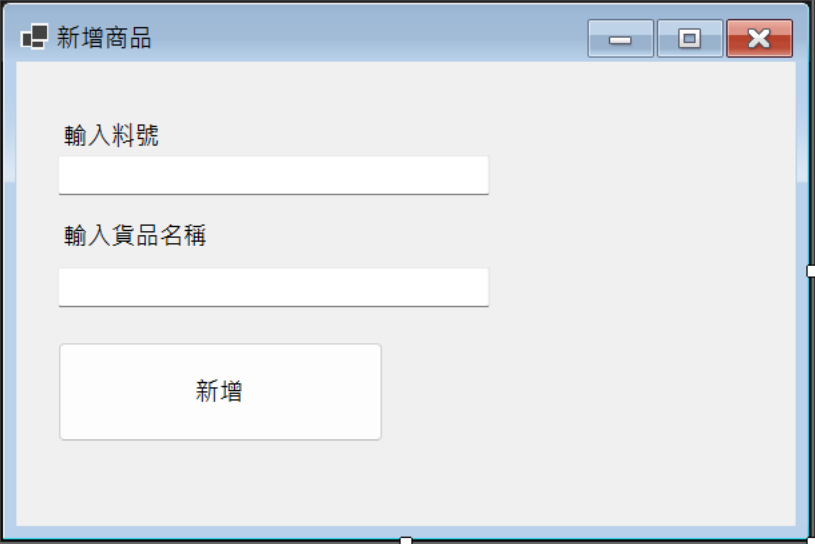
在 BuildSchoolBizSystem 新增一個專案

- 方案名稱：BuildSchoolBizSystem
- 專案名稱：BuildSchoolBizApp
- 範本：Windows Forms

主畫面 v 1.0



新增貨品畫面 -- AddProductForm



The screenshot shows a Windows-style dialog box titled "新增商品" (Add Product). It contains two text input fields and a button. The first field is labeled "輸入料號" (Enter Material Number) and the second is labeled "輸入貨品名稱" (Enter Product Name). Below these fields is a button labeled "新增" (Add).

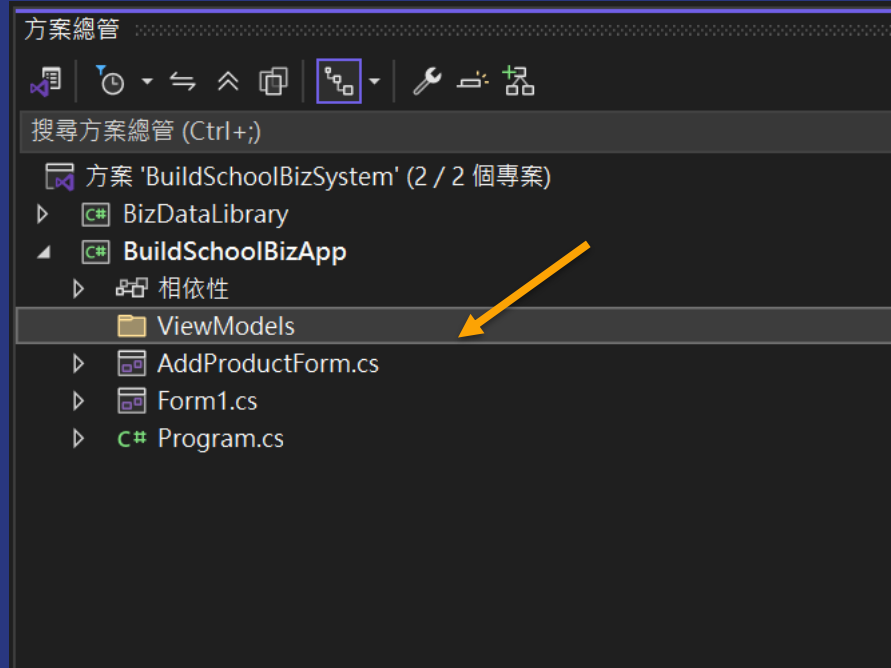
新增商品

輸入料號

輸入貨品名稱

新增

建立一個 ViewModels 資料夾



- (1) 在 ViewModels 目錄下建立一個 ProductViewModel 類別
- (2) 這個類別是為了對應給 AddProductForm 使用的資料

```
public class ProductViewModel
{
    /// <summary>
    /// 貨品的料號，不可以重複
    /// </summary>
    public string? PartNo { get; set; }

    /// <summary>
    /// 貨品的名稱
    /// </summary>
    public string? PartName { get; set; }
}
```


討論

- 為什麼我們需要
ViewModel，直接用
Model 不是很開心嗎？



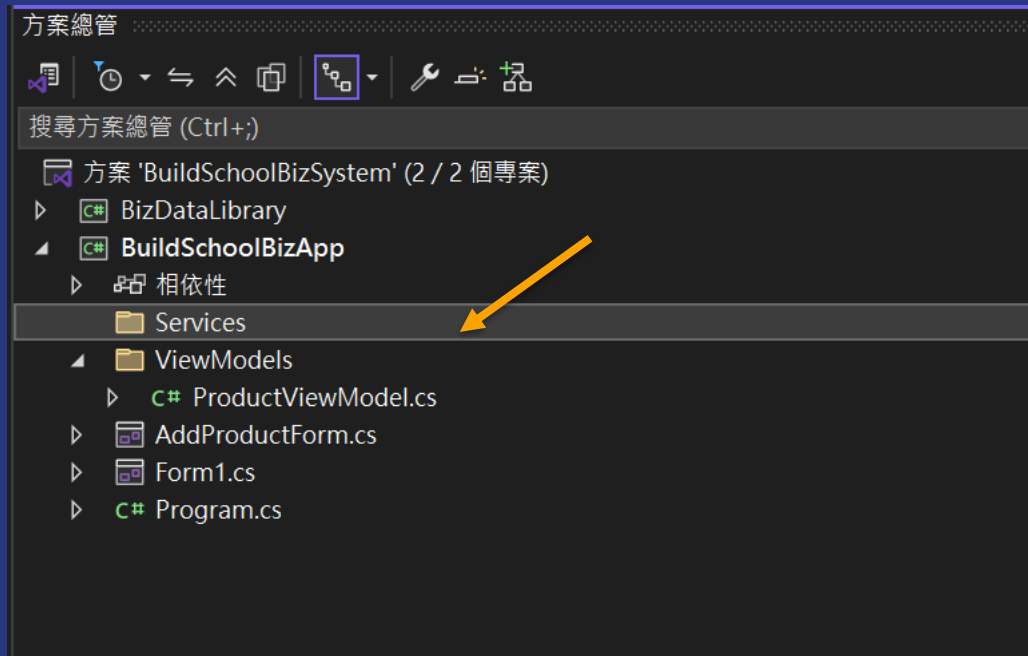
- 在 Form1 中為新增貨品按鈕加入委派函式，用以啟動 AddProductForm

```
private void button1_Click(object sender, EventArgs e)
{
    new AddProductForm().ShowDialog();
}
```

- 為 AddProductForm 的 button 加上程式碼

```
private void button1_Click(object sender, EventArgs e)
{
    //我是因為投影片放不下才切兩行的，你可以不用這麼做
    if (string.IsNullOrEmpty(textBox1.Text) ||
        string.IsNullOrEmpty(textBox2.Text))
    {
        MessageBox.Show("料號或名稱不可為空白");
    }
    else
    {
        var viewModel = new ProductViewModel
        {
            PartNo = textBox1.Text,
            PartName = textBox2.Text
        };
    }
}
```

建立一個 Services 目錄



建立一個給 Service 用於回傳的型別

- 建立 `OperationResult Class`
- 目的是為了回傳操作是否成功，以及發生問題時的例外內容
- 建立 `OperationResultHelper`，用來將log寫入 1 文字檔
 - 註：在真實世界的專案中，很有可能會指定使用某些特定的 `log framework`，例如 `log4net`、`NLog` 之類的。

(1) 在 Services 目錄加入以下兩個類別

```
public class OperationResult
{
    public bool IsSuccessful { get; set; }
    public Exception? Exception { get; set; }
}

public static class OperationResultExtensions
{
    public static string WriteLog(this OperationResult result)
    {
        if (result.Exception != null)
        {
            string path = $"{DateTime.Now.ToString("yyyyMMdd-HH:mm:ss")}_log.txt";
            File.WriteAllText(path, result.Exception.ToString());
            return path;
        }
        else
        {
            return "沒有存檔";
        }
    }
}
```

建立一個 ProductService

- 在 `Services` 目錄下建立 `ProductService class`
- 並且先給它一個假的行為。
- 記住，到目前為止，這個 `Windows Forms` 專案還不曉得如何從資料庫取得真實資料

- (1) Step 1
- (2) 在 Services 目錄加入 ProductService class
- (3) 建立一個靜態的 List<ProductViewModel> 欄位 fakeProducts，作為還沒連上資料庫前測試使用

```
public class ProductService
{
    public static List<ProductViewModel> fakeProducts
        = new List<ProductViewModel>();
}
```


Step 2

(1) 在 ProductService class 建立 Create 方法 (這是假的，只是先測測看)

```
public class ProductService
{
    public static List<ProductViewModel> fakeProducts = new List<ProductViewModel>();

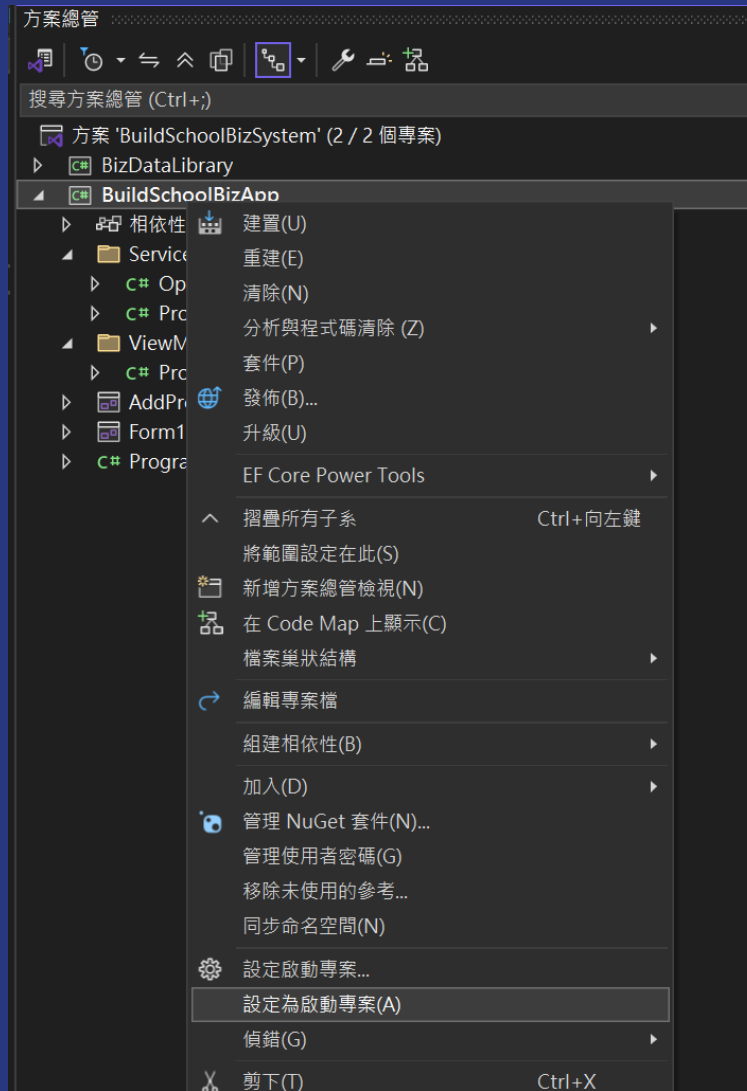
    public OperationResult Create(ProductViewModel input)
    {
        var result = new OperationResult();
        try
        {
            if (fakeProducts.Any(p => p.PartNo == input.PartNo))
            {
                throw new ArgumentException($"料號 {input.PartNo} 已存在");
            }
            fakeProducts.Add(input);
            result.IsSuccessful = true;
        }
        catch (Exception ex)
        {
            result.IsSuccessful = false;
            result.Exception = ex;
        }
        return result;
    }
}
```

Step 3

(1) 修改 AddProductForm class 的程式碼

```
private void button1_Click(object sender, EventArgs e)
{
    //我是因為投影片放不下才切兩行的，你可以不用這麼做
    if (string.IsNullOrEmpty(textBox1.Text) ||
        string.IsNullOrEmpty(textBox2.Text))
    {
        MessageBox.Show("料號或名稱不可為空白");
    }
    else
    {
        var viewModel = new ProductViewModel
        {
            PartNo = textBox1.Text,
            PartName = textBox2.Text
        };
        var service = new ProductService();
        var result = service.Create(viewModel);
        if (result.IsSuccessful)
        {
            MessageBox.Show("新增成功");
        }
        else
        {
            var path = result.WriteLog();
            MessageBox.Show($"新增失敗 請參考 {path}");
        }
    }
}
```

將 BuildSchoolBizApp 設定為起動專案



執行的方式

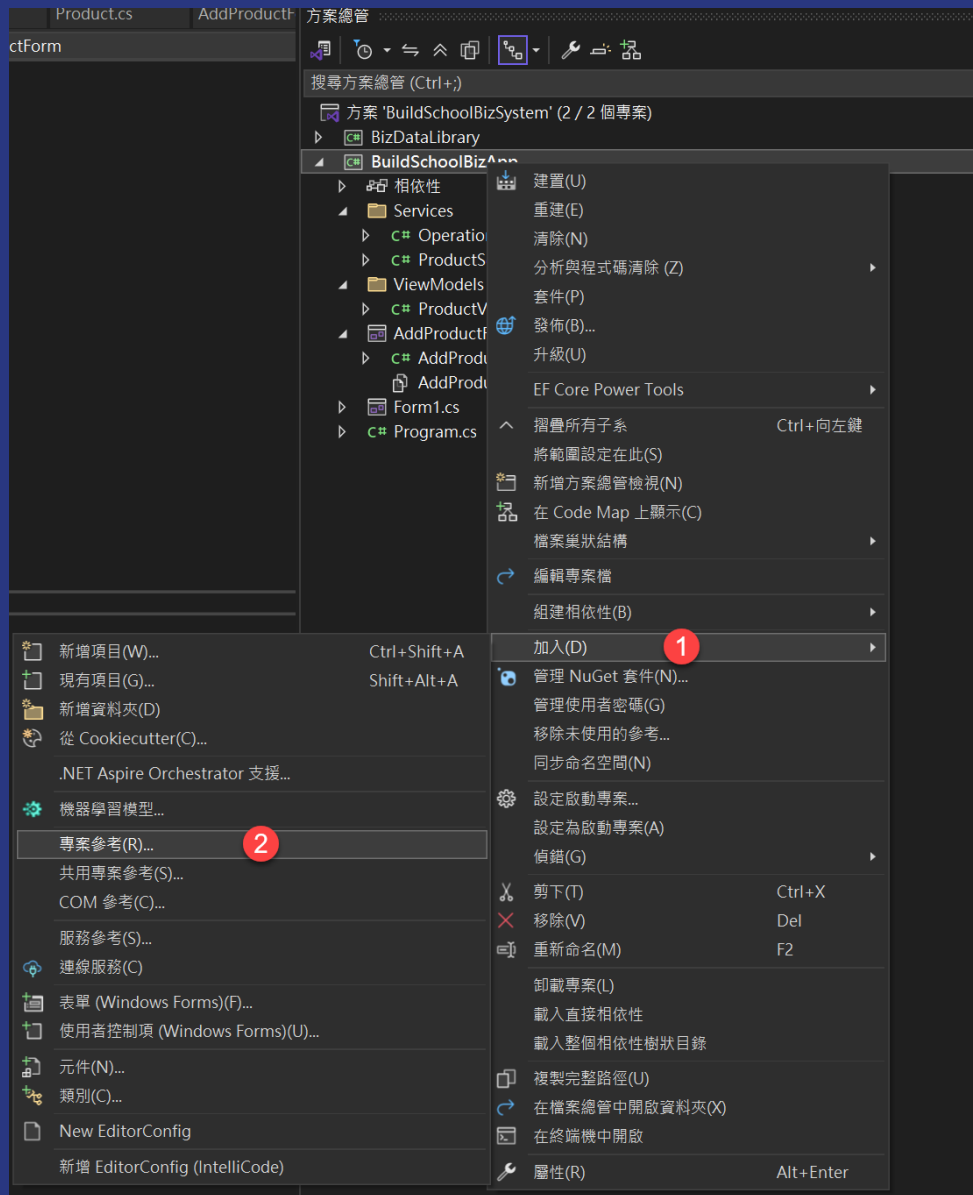
- 呼叫出新增貨品視窗
- 加入一個貨品，接著不要修改 **Textbox** 的內容，直接再按一次新增，觀察是否會有 錯誤提示訊息與產生紀錄檔。

執行

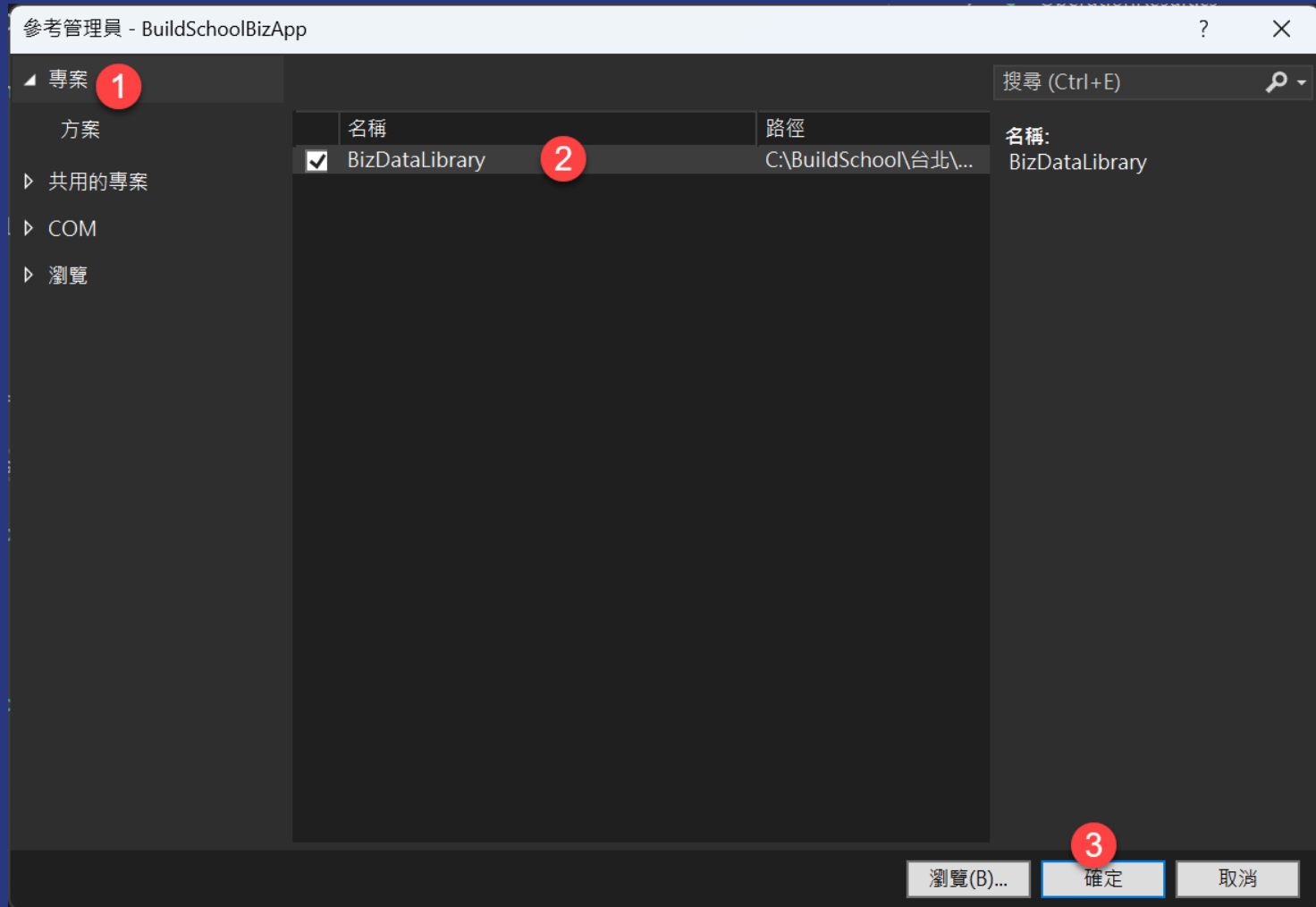
LAB

加入 BizDataLibrary 參考

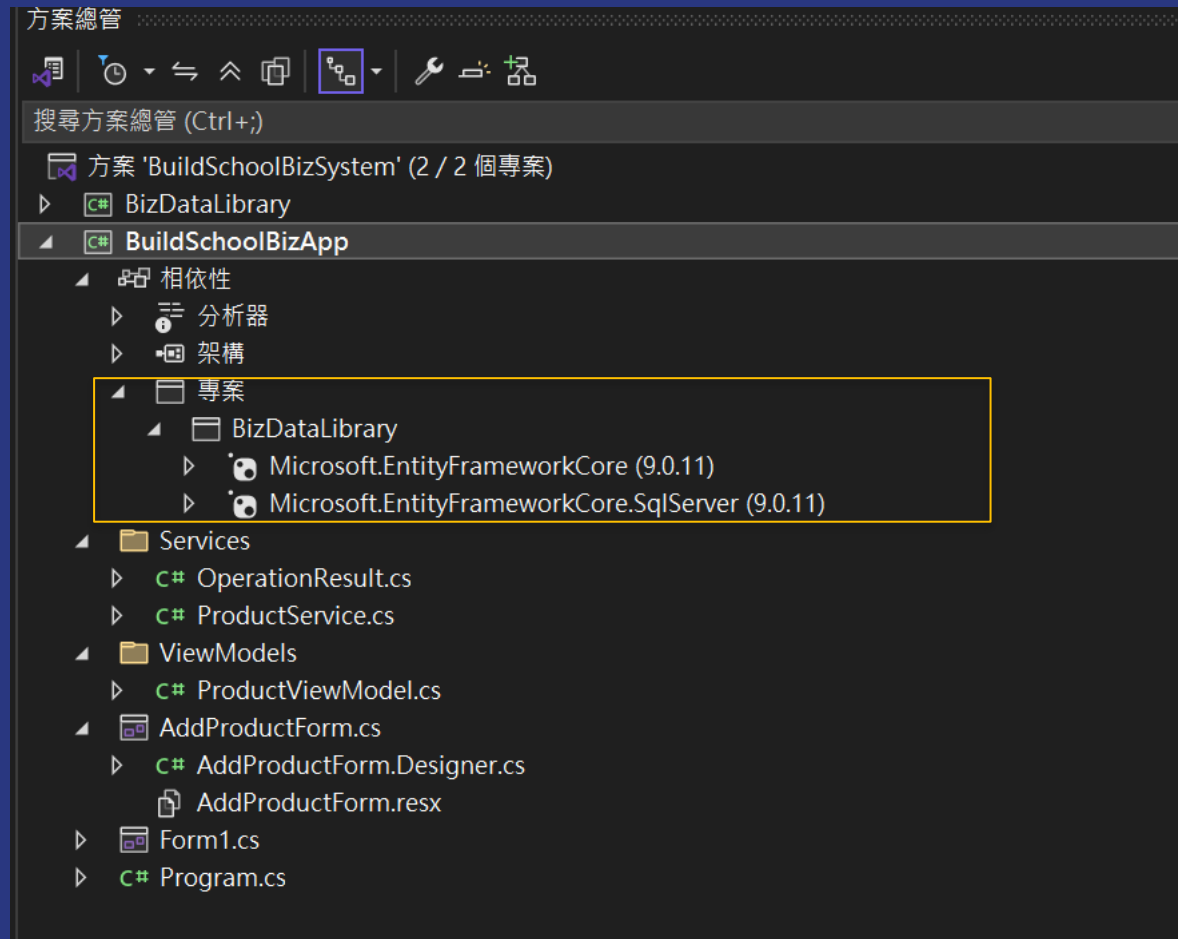
step 1



step 2



step 3 檢查是否安裝完成

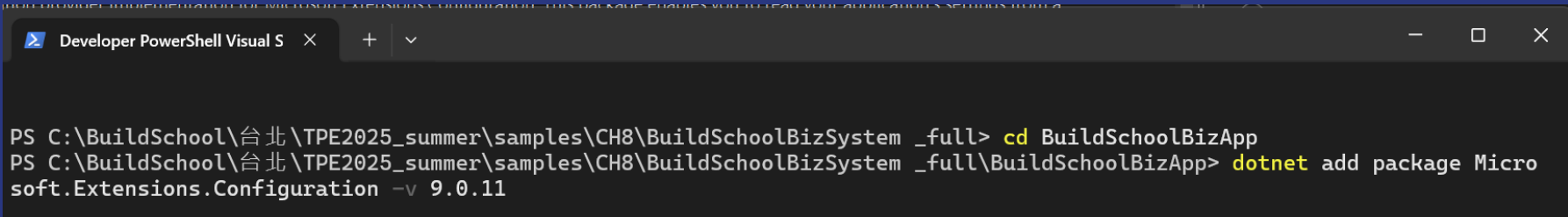


LAB

使用 CLI
加入必要的 nuget package

安裝必要的 nuget package

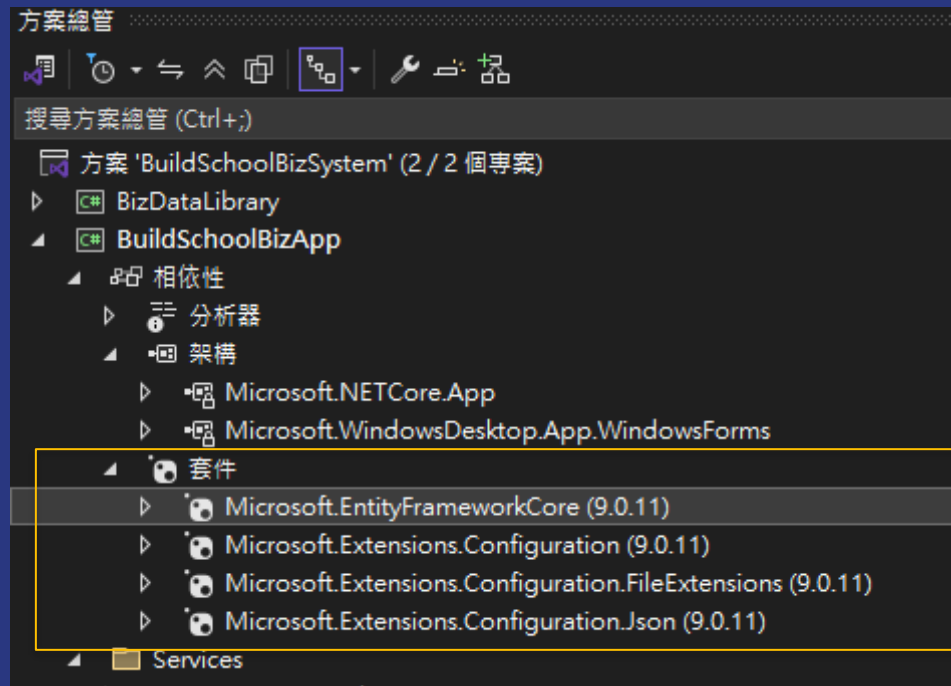
- Microsoft.Extensions.Configuration
- Microsoft.Extensions.Configuration.FileExtensions
- Microsoft.Extensions.Configuration.Json
- Microsoft.EntityFrameworkCore
- 版本 9.0.11
 - 因為框架選 .NET 9
- cli 命令
 - dotnet add package <PackageName> --version <Version>
 - dotnet add package <PackageName> -v <Version>



```
Developer PowerShell Visual S  x  +  v  -  □  x

PS C:\BuildSchool\台北\TPE2025_summer\samples\CH8\BuildSchoolBizSystem_full> cd BuildSchoolBizApp
PS C:\BuildSchool\台北\TPE2025_summer\samples\CH8\BuildSchoolBizSystem_full\BuildSchoolBizApp> dotnet add package Microsoft.Extensions.Configuration -v 9.0.11
```

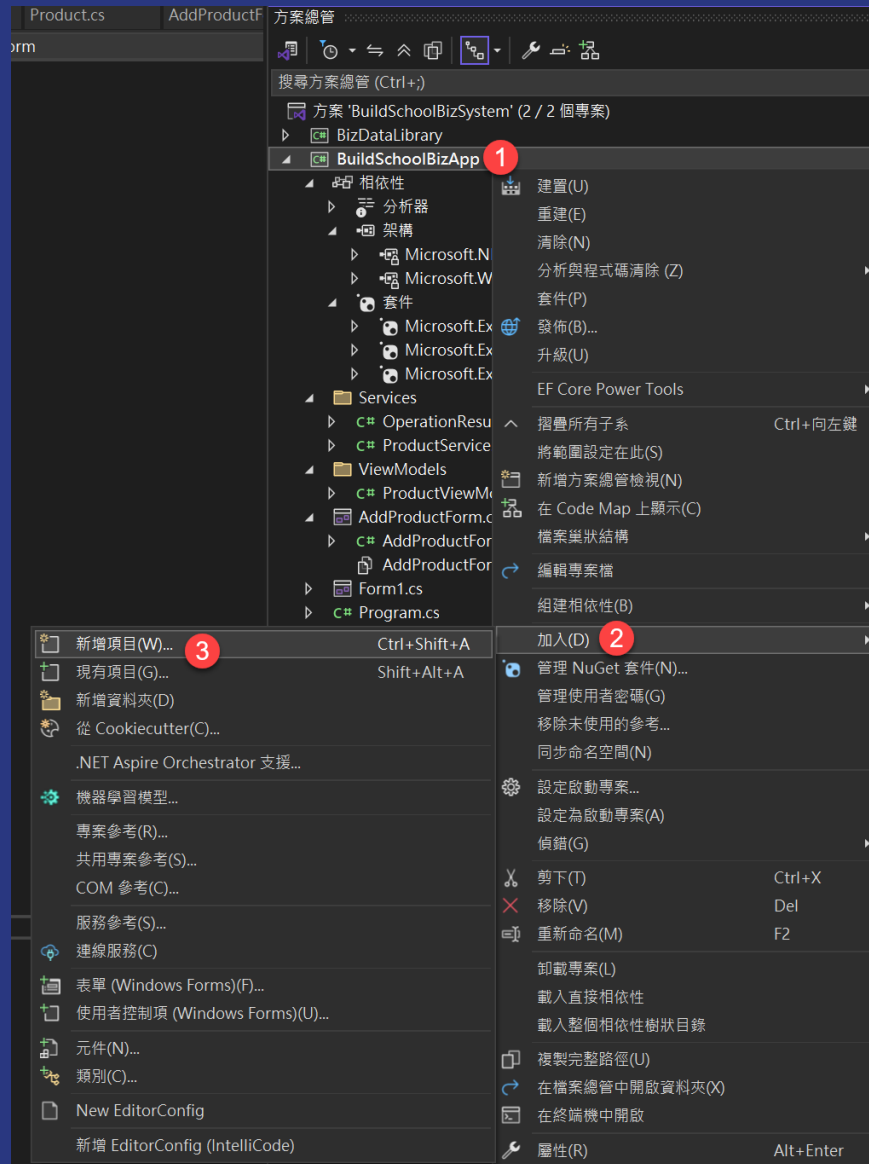
檢查是否安裝完成



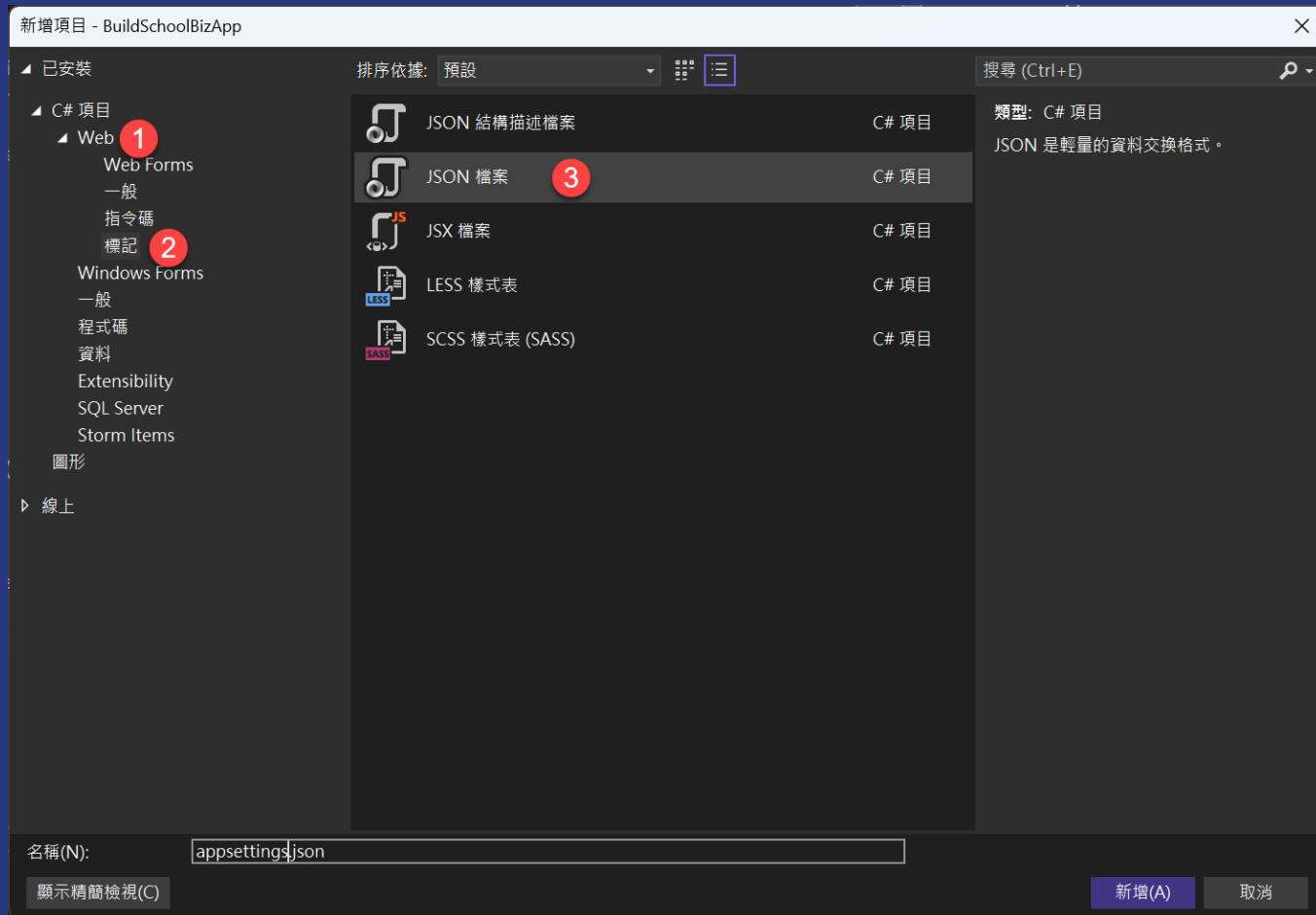
LAB

加入連線字串設定檔

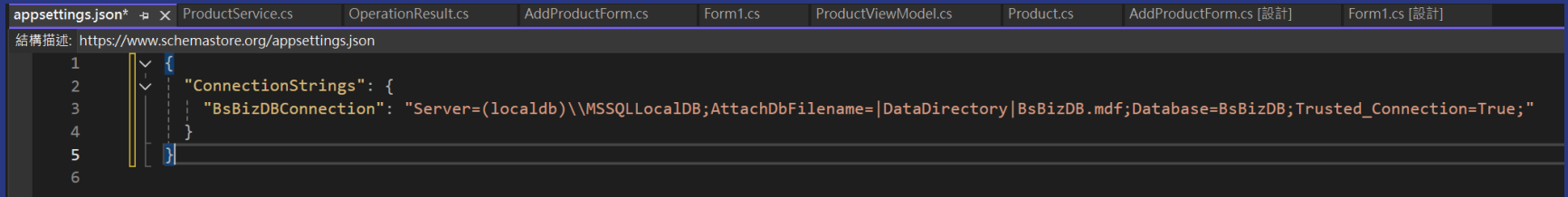
加入 appsettings.json



選【Web】【標記】【JSON 檔案】
命名為 appsettings.json



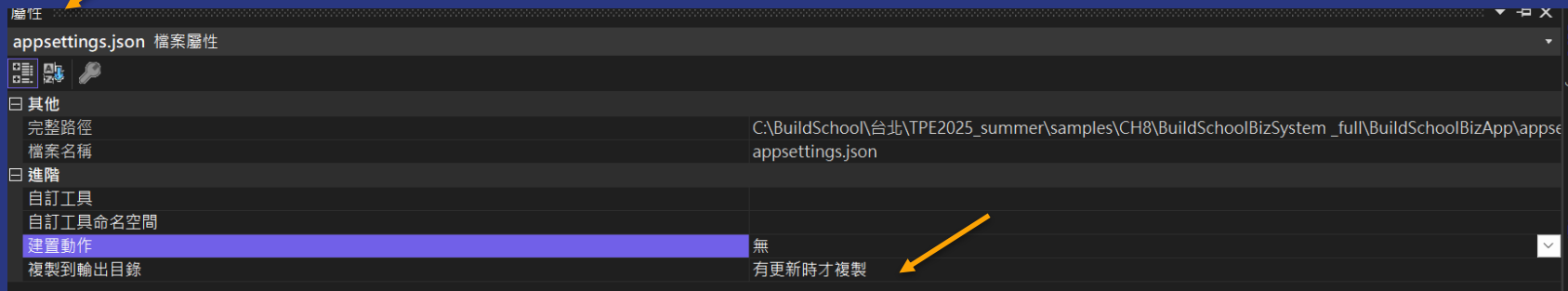
在 appsettings.json 加入正式資料庫連線字串
(完整內容參考 Scripts.txt 內容)



The screenshot shows a code editor with a dark theme. The top tab bar displays several files: appsettings.json*, ProductService.cs, OperationResult.cs, AddProductForm.cs, Form1.cs, ProductViewModel.cs, Product.cs, AddProductForm.cs [設計], and Form1.cs [設計]. The main editor area shows the content of appsettings.json, which is a JSON object with a "ConnectionStrings" property. The value of "ConnectionStrings" is an object containing a single property "BsBizDBConnection" with a long string value representing a database connection string. The string includes a server name, file name, database name, and a trusted connection flag. The code is formatted with syntax highlighting, and a vertical line is visible at the end of the file.

```
1 {  
2   "ConnectionStrings": {  
3     "BsBizDBConnection": "Server=(localdb)\\MSSQLLocalDB;AttachDbFilename=|DataDirectory|BsBizDB.mdf;Database=BsBizDB;Trusted_Connection=True;"  
4   }  
5 }  
6
```


把 appsettings.json 的【複製到輸出目錄】
設定為“有更新才複製”



LAB

初始化資料庫

在 Program class 建立讀取設定檔方法

```
private static IConfiguration LoadConfiguration()
{
    return new ConfigurationBuilder()
        .SetBasePath(Directory.GetCurrentDirectory())
        .AddJsonFile("appsettings.json", optional: true, reloadOnChange: true)
        .Build();
}
```

在 Program class 建立產生 DbContext 方法

```
internal static BizContext CreateBizContext()
{
    var optionsBuilder = new DbContextOptionsBuilder<BizContext>();
    optionsBuilder.UseSqlServer(LoadConfiguration()
        .GetConnectionString("BsBizDBConnection"));
    return new BizContext(optionsBuilder.Options);

    /* 或者使用以下寫法
        return new BizContext(new DbContextOptionsBuilder<BizContext>().UseSqlServer(LoadCo
nfiguration().GetConnectionString("BizDatabase")).Options);
    */
}
```

在 Program class 建立初始化資料庫方法

```
private static void InitialDB()
{
    using(var context = CreateBizContext())
    {
        context.Database.EnsureCreated();
    }
}
```

修改 Program.Main 方法，呼叫必要的初始化

```
static void Main()
{
    ApplicationConfiguration.Initialize();
    → InitialDB();
    Application.Run(new Form1());
}
```

LAB

將 Windows Forms 和
BizDataLibrary 結合

Step 1 修改 ProductService Class ,
透過 BizRepository (使用建構子注入) 和資料庫連接
(為什麼此時已經不需要檢查 key 重複的問題呢?)

```
public class ProductService
{
    private readonly BizRepository _repository;
    public ProductService(BizRepository repository)
    {
        _repository = repository;
    }
    public OperationResult Create(ProductViewModel input)
    {
        var result = new OperationResult();
        try
        {
            var entity = new Product
            {
                PartNo = input.PartNo!,
                PartName = input.PartName!
            };
            _repository.Create(entity);
            _repository.SaveChanges();
            result.IsSuccessful = true;
        }
        catch (Exception ex)
        {
            result.IsSuccessful = false;
            result.Exception = ex;
        }
        return result;
    }
}
```


Step 2 修改 AddPeoductForm 中呼叫 ProductService 部分

```
private void button1_Click(object sender, EventArgs e)
{
    //我是因為投影片放不下才切兩行的，你可以不用這麼做
    if (string.IsNullOrEmpty(textBox1.Text) ||
        string.IsNullOrEmpty(textBox2.Text))
    {
        MessageBox.Show("料號或名稱不可為空白");
    }
    else
    {
        var viewModel = new ProductViewModel
        {
            PartNo = textBox1.Text.Trim(),
            PartName = textBox2.Text.Trim()
        };
        using var context = Program.CreateBizContext();
        var service = new ProductService(new BizRepository(context));
        var result = service.Create(viewModel);
        if (result.IsSuccessful)
        {
            MessageBox.Show("新增成功");
        }
        else
        {
            var path = result.WriteLog();
            MessageBox.Show($"新增失敗 請參考 {path}");
        }
    }
}
```

執行

LAB

繼續完成其他的新增作業

AddSalesmanForm (新增銷售人員資料)



The image shows a Windows-style dialog box titled "新增銷售人員" (Add Salesman). The window has a standard title bar with minimize, maximize, and close buttons. Inside the window, there is a text input field with the placeholder text "輸入業務員姓名" (Enter Salesman Name). Below the input field is a button labeled "新增" (Add).

新增銷售人員

輸入業務員姓名

新增

Step 1

(1) 新增 SalesmanViewModel

```
public class SalesmanViewModel
{
    public int JobNumber { get; set; }

    public string? Name { get; set; }
}
```

Step 2 新增 SalesmanService

```
public class SalesmanService
{
    private readonly BizRepository _repository;
    public SalesmanService(BizRepository repository)
    {
        _repository = repository;
    }

    public OperationResult Create(SalesmanViewModel input)
    {
        var result = new OperationResult();
        try
        {
            var entity = new Salesman
            {
                JobNumber = input.JobNumber!,
                Name = input.Name!
            };
            _repository.Create(entity);
            _repository.SaveChanges();
            result.IsSuccessfull = true;
        }
        catch (Exception ex)
        {
            result.IsSuccessfull = false;
            result.Exception = ex;
        }
        return result;
    }
}
```

Step 3在 Form1 中為新增銷售人員按鈕加入委派函式，用以啟動 AddSalesmanForm

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private void button1_Click(object sender, EventArgs e)
    {
        new AddProductForm().ShowDialog();
    }

    private void button2_Click(object sender, EventArgs e)
    {
        new AddSalesmanForm().ShowDialog();
    }
}
```

Step 4 為 AddSalesmanForm 的按鈕加入委派函式

```
private void button1_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(textBox1.Text))
    {
        MessageBox.Show("業務員姓名不可為空白");
        return;
    }
    var viewModel = new SalesmanViewModel
    {
        Name = textBox1.Text.Trim()
    };
    using var context = Program.CreateBizContext();
    var service = new SalesmanService(new BizRepository(context));
    var result = service.Create(viewModel);
    if (result.IsSuccessful)
    {
        MessageBox.Show("新增成功");
    }
    else
    {
        var path = result.WriteLog();
        MessageBox.Show($"新增失敗 請參考 {path}");
    }
}
```


執行

AddProcurementForm (新增進貨資料)

新增進貨資料

貨品選擇

listBox1

進貨數量

0

單價

0

日期

2025年11月20日

新增

Step 1

(1) 新增 ProcurementViewModel

```
public class ProcurementViewModel
{
    public int ProcurementId { get; set; }

    public string? PartNo { get; set; }

    public DateTime PurchasingDay { get; set; }

    public int Quantity { get; set; }

    public int UintPrice { get; set; }

    public int InvetoryQuantity { get; set; }
}
```

Step 2

(1) 新增 ProcurementService class

```
public class ProcurementService
{
    private readonly BizRepository _repository;
    public ProcurementService(BizRepository repository)
    {
        _repository = repository;
    }

    public OperationResult Create(ProcurementViewModel input)
    {
        var result = new OperationResult();
        try
        {
            var entity = new Procurement
            {
                PartNo = input.PartNo!,
                Quantity = input.Quantity,
                InventoryQuantity = input.InventoryQuantity,
                UnitPrice = input.UnitPrice,
                PurchasingDay = input.PurchasingDay
            };
            _repository.Create(entity);
            _repository.SaveChanges();
            result.IsSuccessfull = true;
        }
        catch (Exception ex)
        {
            result.IsSuccessfull = false;
            result.Exception = ex;
        }
        return result;
    }
}
```

Step 3 因為 ListBox 需要貨品的基本資料，修改 **ProductService** 加入以下方法

```
public IEnumerable<ProductViewModel> GetAll()
{
    foreach (var item in _repository.GetAll<Product>().OrderBy(x=> x.PartNo))
    {
        yield return new ProductViewModel
        {
            PartNo = item.PartNo,
            PartName = item.PartName
        };
    }
}
```

Step 4 我們想在 ListBox 顯示 {PartNo} – {PartName}
所以在 ProductViewModel 新增一個屬性 DisplayName

```
public class ProductViewModel
{
    public string? PartNo { get; set; }

    public string? PartName { get; set; }

    public string DisplayName => $"{PartNo} - {PartName}"; ←
}
```

Step 5 為 AddProcurementForm 的 Load 事件撰寫委派函式 (在設計畫面上，選擇 Form 按兩下滑鼠左鍵)

```
private void AddProcurementForm_Load(object sender, EventArgs e)
{
    using var context = Program.CreateBizContext();
    var service = new ProductService(new BizRepository(context));
    var products = service.GetAll().ToList();
    listBox1.DataSource = products;
    listBox1.DisplayMember = "DisplayName";
    listBox1.ValueMember = "PartNo";
}
```

Step 6 為 AddProcurementForm 的新增進貨按鈕 click 事件撰寫委派函式

```
private void button1_Click(object sender, EventArgs e)
{
    if (listBox1.SelectedIndex == -1)
    {
        MessageBox.Show("請選擇產品");
        return;
    }
    var viewModel = new ProcurementViewModel
    {
        PartNo =(string)listBox1.SelectedValue!,
        PurchasingDay = dateTimePicker1.Value,
        Quantity = (int)numericUpDown1.Value,
        InventoryQuantity = (int)numericUpDown1.Value,
        UintPrice = (int)numericUpDown2.Value
    };
    using var context = Program.CreateBizContext();
    var service = new ProcurementService(new BizRepository(context));
    var result = service.Create(viewModel);
    if (result.IsSuccessful)
    {
        MessageBox.Show("新增成功");
    }
    else
    {
        var path = result.WriteLog();
        MessageBox.Show($"新增失敗 請參考 {path}");
    }
}
```


Step 7 在 Form1 中為新增進貨資料按鈕加入委派函式，用以啟動 AddProcurementForm

```
private void button3_Click(object sender, EventArgs e)
{
    new AddProcurementForm().ShowDialog();
}
```

AddSellingForm (新增出貨資料)

The screenshot shows a Windows-style application window titled "銷售資料" (Sales Data). The window contains the following elements:

- 貨品選擇 (Product Selection):** A label above a large empty text box labeled "listBox1".
- 出貨數量 (Shipping Quantity):** A label above a `NumericUpDown` control showing the value "1".
- 單價 (Unit Price):** A label above another `NumericUpDown` control showing the value "1".
- 銷售人員 (Salesperson):** A label above a large empty text box labeled "listBox2".
- 日期 (Date):** A label above a date picker control showing "2025年11月21日".
- 新增 (Add):** A rectangular button located at the bottom center of the form.

兩個 `NumericUpDown` 的 `Minimum` 屬性值改為 1

Step 1 新增 SellingViewModel

```
public class SellingViewModel
{
    public int SellingId { get; set; }

    public int SalesJobNumber { get; set; }

    public string? PartNo { get; set; }

    public DateTime SellingDay { get; set; }

    public int Quantity { get; set; }

    public int UnitPrice { get; set; }
}
```

Step 2 因為 ListBox (銷售員) 需要銷售員的基本資料，所以需要修改 SalesmanService 加入以下方法

[跟 ProductService.GetAll 比較一下寫法]

```
public IEnumerable<SalesmanViewModel> GetAll()
{
    return _repository.GetAll<Salesman>()
        .Select(s => new SalesmanViewModel
        {
            JobNumber = s.JobNumber,
            Name = s.Name
        }).AsEnumerable();
}
```

Step 3 我們想在 ListBox 顯示 {JobNumber} - {Name}
所以在 SalesmanViewModel 新增一個屬性 DisplayName

```
public class SalesmanViewModel
{
    public int JobNumber { get; set; }

    public string? Name { get; set; }

    public string? DisplayName => $"{JobNumber} - {Name}"; ←
}
```

Step 4

(1) 為 AddSellingForm 的 Load 事件加入委派函式

(2) 加入繫結資料到兩個 ListBox 的函式

```
private void AddSellingForm_Load(object sender, EventArgs e)
{
    using var context = Program.CreateBizContext();
    BindProductListBox(context);
    BindSalesmanListBox(context);
}

private void BindProductListBox(DbContext context)
{
    var service = new ProductService(new BizRepository(context));
    var products = service.GetAll().ToList();
    listBox1.DataSource = products;
    listBox1.DisplayMember = "DisplayName";
    listBox1.ValueMember = "PartNo";
}

private void BindSalesmanListBox(DbContext context)
{
    var service = new SalesmanService(new BizRepository(context));
    var salesmen = service.GetAll().ToList();
    listBox2.DataSource = salesmen;
    listBox2.DisplayMember = "DisplayName";
    listBox2.ValueMember = "JobNumber";
}
```

如何決定可以售出

- 當 `user` 要賣出貨品之前，必須先確定庫存是足夠的。

Step 5 在 ProcurementService 新增取得目前庫存的方法

```
public int GetInventoryQuantity(string partNo)
{
    return _repository.GetAll<Procurement>()
        .Where(p => p.PartNo == partNo && p.InventoryQuantity > 0)
        .Sum(p => p.InventoryQuantity);
}
```


SellingService 的重點

- 和之前不同了，不僅要新增資料到 Selling 資料表，
- 還要扣除進貨資料中的庫存數 (InventoryQuantity)
- 同時也要新增資料到 SellingSource 資料表

step 6 新增 SellingService class

```
public class SellingService
{
    private readonly BizRepository _repository;
    public SellingService(BizRepository repository)
    {
        _repository = repository;
    }
}
```

流程分析

1. 新增資料到 `Selling`，必須 `SaveChanges` 才能拿到新的 `SellingId`
2. 扣除庫存 (`Procurement.InventoryQuantity`)，同時在扣除一筆庫存時，需增加一筆 `SellingSource` 紀錄
3. 組合上述流程，並且透過 `Transaction` 管理異動過程

step 7 先完成流程 1

在 SellingService class 加入以下程式碼 Create 方法

```
private Selling CreatSelling(SellingViewModel input)
{
    Selling entity = new Selling()
    {
        PartNo = input.PartNo!,
        Quantity = input.Quantity,
        SalesJobNumber = input.SalesJobNumber,
        SellingDay = input.SellingDay,
        UnitPrice = input.UnitPrice,
    };
    _repository.Create(entity);
    // 要先 SaveChanges , 才能拿到 Selling.SellingId
    _repository.SaveChanges();
    return entity;
}
```

流程 2 的細節 (1)

- 執行流程 1 以取得 `Selling.SellingId`
- 從 `Procurement` 取得所有庫存 (`InventoryQuantity`) 大於 0 的資料，並按照採購日期時間排列 (先進先出)。

流程 2 的細節 (2)

- 依序取出上述資料
 - 如果該筆資料庫存 大於等於 尚未扣除的出貨量 (`sellCount` 參數)，則將該筆資料庫存扣除 `sellCount`，新增資料到 `SellingSource` (數量剛好等於 `sellCount`)，並且將 `sellCount` 設定為 0
 - 如果尚未扣除的出貨量 (`sellCount` 參數) 大於該筆資料庫存，則 `sellCount = sellCount - 該筆庫存`，新增資料到 `SellingSource` (數量剛好等於 該筆庫存)
 - 當 `sellCount <= 0` 離開迴圈

step 8 完成流程 2

在 SellingService class 加入以下程式碼 (這一頁和下一頁)

```
private void CreateSellingSource(int sellingId, int procurementId, int sellQuantity)
{
    SellingSource entity = new SellingSource()
    {
        SellingId = sellingId,
        ProcurementId = procurementId,
        Quantity = sellQuantity
    };
    _repository.Create(entity);
}
```

```
private void UpdateProcurementInventory(SellingViewModel input)
{
    Selling entity = CreatSelling(input);
    var remainQuantity = input.Quantity;
    var procurements = _repository.GetAll<Procurement>()
        .Where(p => p.PartNo == input.PartNo && p.InventoryQuantity > 0)
        .OrderBy(p => p.PurchasingDay);
    foreach (var p in procurements)
    {
        if (remainQuantity <= 0)
        {
            break;
        }
        if (p.InventoryQuantity >= remainQuantity)
        {
            // 出貨量小於等於庫存量
            CreateSellingSource(entity.SellingId, p.ProcurementId, remainQuantity);
            p.InventoryQuantity -= remainQuantity;
            remainQuantity = 0;
        }
        else
        {
            // 出貨量大於庫存量
            CreateSellingSource(entity.SellingId, p.ProcurementId, p.InventoryQuantity);
            remainQuantity -= p.InventoryQuantity;
            p.InventoryQuantity = 0;
        }
    }
}
```


流程 3

- 組合上述流程，並且透過 Transaction 管理異動過程

step 9 加入異動管理

在 BizRepository class 加入以下方法 BeginTransaction

```
public IDbContextTransaction BeginTransaction()
{
    return _context.Database.BeginTransaction();
}
```

step 10 完成流程 3

在 SellingService class 加入以下程式碼 Create 方法

```
public OperationResult Create(SellingViewModel input)
{
    var result = new OperationResult();
    // 使用交易處理，確保資料一致性，這裡採用舊式的 using 語法
    using (var transaction = _repository.BeginTransaction())
    {
        try
        {
            UpdateProcurementInventory(input);
            _repository.SaveChanges();
            transaction.Commit();
            result.IsSuccessful = true;
        }
        catch (Exception ex)
        {
            transaction.Rollback();
            result.IsSuccessful = false;
            result.Exception = ex;
        }
    }
    return result;
}
```

step 11 在AddSellingForm 加入庫存檢查

```
private static bool CanSell(string partNo, int quantity)
{
    using var context = Program.CreateBizContext();
    var service = new ProcurementService(new BizRepository(context));
    var inventoryQuantity = service.GetInventoryQuantity(partNo);
    return inventoryQuantity >= quantity;
}
```

step 12 完成 AddSellingForm

為 AddSellingForm 加入新增按鈕的委派函式
程式碼太多，放在下一頁

```
private void button1_Click(object sender, EventArgs e)
{
    if (listBox1.SelectedIndex == -1)
    {
        MessageBox.Show("請選擇產品");
        return;
    }
    if (listBox2.SelectedIndex == -1)
    {
        MessageBox.Show("請選擇業務員");
        return;
    }
    var partNo = (string)listBox1.SelectedValue!;
    var quantity = (int)numericUpDown1.Value;

    if (!CanSell(partNo, quantity))
    {
        MessageBox.Show("庫存不足，無法銷售");
        return;
    }
    var viewmodel = new SellingViewModel()
    {
        PartNo = partNo, Quantity = quantity, SalesJobNumber = (int)listBox2.SelectedValue!,
        SellingDay = dateTimePicker1.Value, UnitPrice = (int)numericUpDown2.Value,
    };
    using var context = Program.CreateBizContext();
    var service = new SellingService(new BizRepository(context));
    var result = service.Create(viewmodel);
    if (result.IsSuccessful)
    {
        MessageBox.Show("銷售成功");
    }
    else
    {
        var path = result.WriteLog();
        MessageBox.Show($"銷售失敗 請參考 {path}");
    }
}
```

step 11 啟動 AddSellingForm

- 在 Form1 中為新增出貨資料按鈕加入委派函式，用以啟動 AddSellingForm

```
private void button4_Click(object sender, EventArgs e)
{
    new AddSellingForm().ShowDialog();
}
```

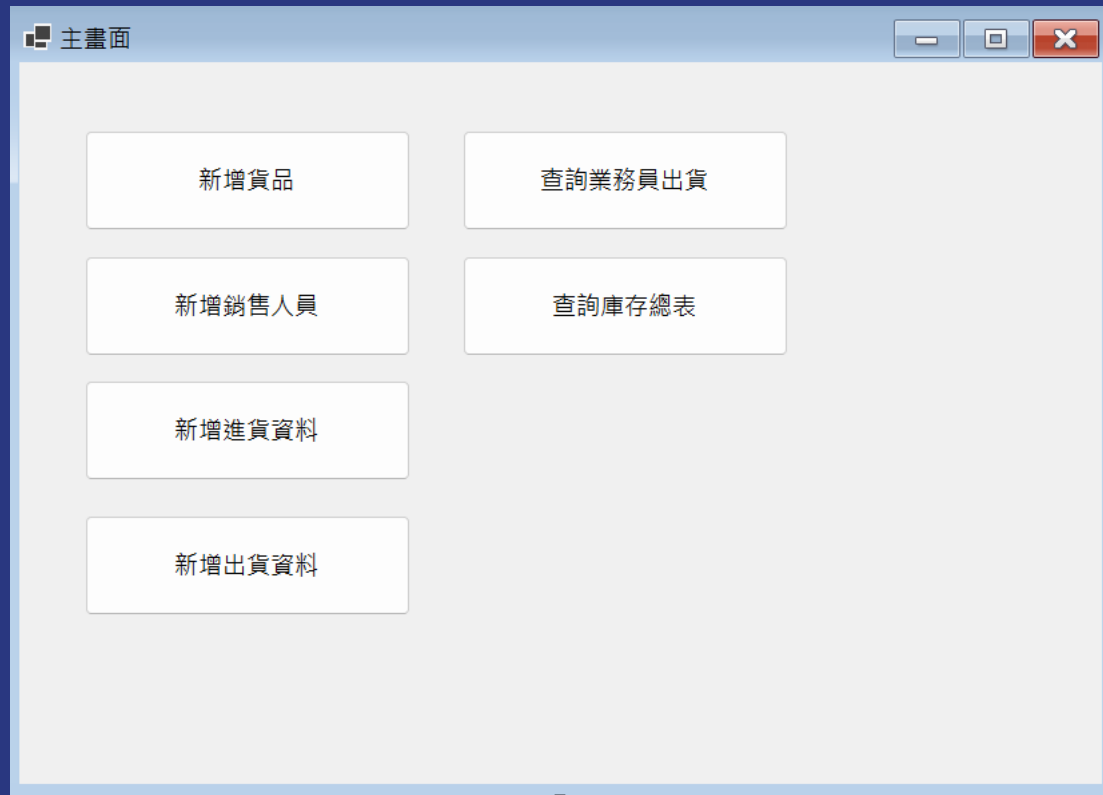


執行

LAB

加入一些查詢功能

主畫面 v 2.0



QuerySellingBySalesForm (查詢銷售人員出貨)

查詢銷售人員出貨

銷售人員

listBox2

出貨日期

From 2025年11月21日

To 2025年11月21日

查詢

這是 DataGridView (Dock Bottom)

Step 1 新增 SellingQueryViewModel

```
public class SellingQueryViewModel
{
    public int SellingId { get; set; }
    public int SalesJobNumber { get; set; }

    public string? SalesName { get; set; }

    public string? PartNo { get; set; }

    public DateTime SellingDay { get; set; }

    public int Quantity { get; set; }

    public int UnitPrice { get; set; }

    public int TotalPrice => Quantity * UnitPrice;
}
```

Step 2 在 SellingService 加入以下方法

```
public IEnumerable<SellingQueryViewModel> GetSellingBySalesAndDay(int jobNumber, DateTime begin, DateTime end)
{
    return from selling in _repository.GetAll<Selling>()
           join sales in _repository.GetAll<Salesman>()
           on selling.SalesJobNumber equals sales.JobNumber
           where selling.SalesJobNumber == jobNumber
              && selling.SellingDay >= begin
              && selling.SellingDay < end
           select new SellingQueryViewModel
           {
               SellingId = selling.SellingId,
               SalesJobNumber = sales.JobNumber,
               SalesName = sales.Name,
               PartNo = selling.PartNo,
               SellingDay = selling.SellingDay,
               Quantity = selling.Quantity,
               UnitPrice = selling.UnitPrice,
           };
}
```

Step 3 在 QuerySellingBySalesForm 加入 Load 的委派函式

```
private void QuerySellingBySalesForm_Load(object sender, EventArgs e)
{
    using var context = Program.CreateBizContext();
    BindSalesmanListBox(context);
}

private void BindSalesmanListBox(DbContext context)
{
    var service = new SalesmanService(new BizRepository(context));
    var salesmen = service.GetAll().ToList();
    listBox1.DataSource = salesmen;
    listBox1.DisplayMember = "DisplayName";
    listBox1.ValueMember = "JobNumber";
}
```

Step 4 在 QuerySellingBySalesForm 加入按鈕的委派函式

```
private void button1_Click(object sender, EventArgs e)
{
    if (listBox1.SelectedIndex == -1)
    {
        MessageBox.Show("請選擇業務員");
        return;
    }
    int jobNumber = (int)listBox1.SelectedValue;
    var begin = dateTimePicker1.Value.Date;
    var end = dateTimePicker2.Value.Date.AddDays(1);
    using var context = Program.CreateBizContext();
    var service = new SellingService(new BizRepository(context));
    var results = service.GetSellingBySalesAndDay(jobNumber, begin, end);
    dataGridView1.DataSource = results.ToList();
}
```

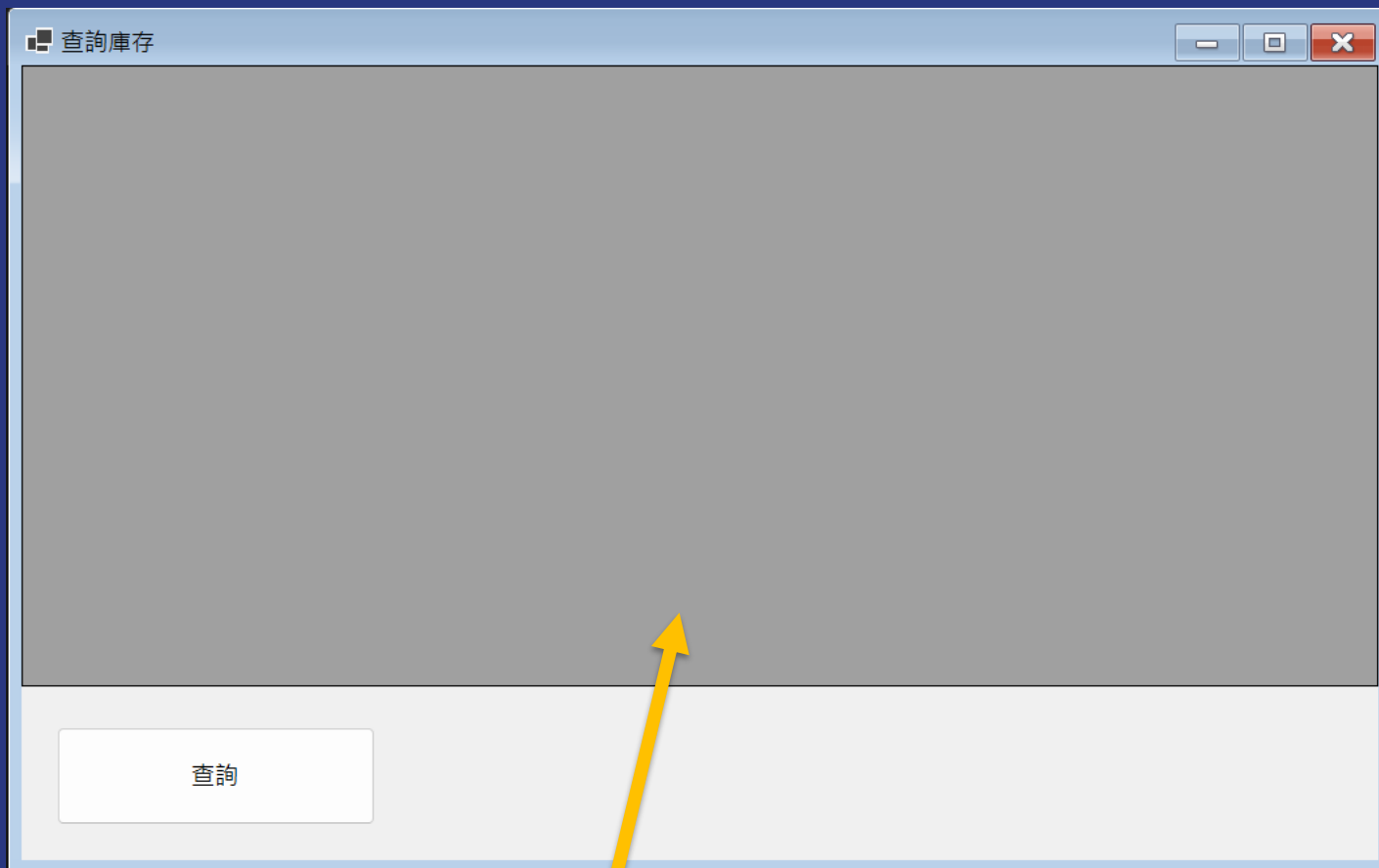
Step 5 在 Form1 中為查詢銷售人員出貨按鈕加入委派函式，用以啟動 QuerySellingBySalesForm

```
private void button5_Click(object sender, EventArgs e)
{
    new QuerySellingBySalesForm().ShowDialog();
}
```




執行

QueryInventoryForm (查詢庫存總表)



這是 DataGridView (Dock Top)

Step 1 新增 InventoryQueryViewModel

```
public class InventoryQueryViewModel
{
    public string? PartNo { get; set; }

    public string? PartName { get; set; }

    public int TotalInventoryQuantity { get; set; }
}
```

Step 2 ProcurementService 加入查詢庫存方法

```
public IEnumerable<InventoryQueryViewModel> GetInventorySummary()
{
    var temp = from p in _repository.GetAll<Product>()
                join q in _repository.GetAll<Procurement>()
                on p.PartNo equals q.PartNo
                select new
                {
                    PartNo = p.PartNo,
                    PartName = p.PartName,
                    InventoryQuantity = q.InventoryQuantity
                };
    return from t in temp
           group t by new { t.PartNo, t.PartName } into g
           select new InventoryQueryViewModel
           {
               PartNo = g.Key.PartNo,
               PartName = g.Key.PartName,
               TotalInventoryQuantity = g.Sum(x => x.InventoryQuantity)
           };
}
```

Step 3 QueryStockForm 加入以下委派函式

```
private void button1_Click(object sender, EventArgs e)
{
    using var context = Program.CreateBizContext();
    var service = new Services.ProcurementService(new BizRepository(context));
    dataGridView1.DataSource = service.GetInventorySummary().ToList();
}
```

Step 4 在 Form1 中為查詢庫存總表按鈕加入委派函式，用以啟動 QueryStockForm

```
private void button6_Click(object sender, EventArgs e)
{
    new QueryInventoryForm().ShowDialog();
}
```

在這裏你將學到

Learn How to Learn

- 學新東西、新技術的能力
- 尋找解答的能力
- 隨時吸取新知識的能力

跟著你一輩子的能力 ...

對學生的好處

不只是就業銜接，培養自我解決問題的能力及信心!



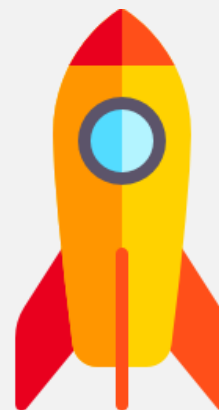
解決問題能力 (Problem-Solving)

- Learn how to learn
- Peer Learning
- Project-based



就業銜接 (Professional skills ready)

- Technical + Soft Skill



擴散及影響 (Make impacts)

- Mentor Networks